

R고싶조. 최종 보고서

R을 통한 영어 단어 시험 및 미니 게임 구현

2072063 식품공학과 김예인 / 2476306 통계학과 조은빈 / 2429016 통계학과
박미진



목차

1. 기획 단계

- 1) MainTest
 - I . MainTest
 - II . ReTest
- 2) MiniGame

2. R 프로그래밍 과정

- [1] R 프로그래밍 과정
 - 1) MainTest
 - I . MainTest
 - II . ReTest
 - 2) MiniGame
 - 3) EngWordTest
- [2] 알고리즘
 - 1) MainTest
 - I . MainTest
 - II . ReTest
 - 2) MiniGame
 - 3) EngWordTest

3. 사용 설명서

4. 마무리 및 해당 프로그램을 통한 영어 학습의 의의

1. 기획 단계

영어 과외를 하는 도중 영단어 시험을 치르며 반복되는 노동을 최소화하기 위해 해당 프로그램을 기획하였다. 조원 중 온라인 화상 영어 과외를 진행했던 학생이 있어 영어 단어 시험을 치르기 위해 필요한 요소를 자문받아 프로그램을 구체화 했다. 영어 단어 시험을 치르기 전에 학생이 스스로 학습 정도를 확인할 수 있고, 영어 단어 시험도 치를 수 있는 단어장을 구현하길 원했으며, 시험을 치를 때 영어 단어의 수도 유동적으로 사용해 DAY별로 단어 수가 다른 점을 보완하길 바랐다. 더욱이 오답률이 높은 경우, 필수적으로 재시험을 실시하여 더 효과가 좋은 학습이 구현 되길 요구했다. 이런 점들을 토대로 영어 단어 시험인 **MainTest**를 구현했으며 이 안에 **ReTest**라는 재시험 또한 같이 구성하였다. 영어 단어를 조금 더 쉽고, 재미있게 학습할 수 있었으면 좋겠다는 의견을 반영하여, **MainTest** 이외에 **MiniGame** 또한 기획하여 영어 단어를 암기하는 학생들에게 조금 더 접근성이 높은 프로그램을 기획했다. 이 프로그램을 통해 학생들이 영어 단어를 효율적으로, 진입 장벽이 낮게 다가갈 수 있길 기대한다.

-1) MainTest

I . MainTest

자동으로 영단어 시험을 치르고 채점해주는 프로그램이다. 단어목록을 엑셀파일로 입력해줄 수 있고, 사용자가 원하는 단어장으로 시험지를 자동으로 생성한다. 이 때 진짜 단어를 암기한 것인지, 순서에 따라 외운 것인지 판가름하기 위하여 무작위로 문제를 출제한다. 시험은 단어의 뜻을 문제로 출력하면 사용자가 영단어를 입력하는 방식으로 진행된다. 시험을 모두 마치면 결과를 집계하여 보여주므로, 사용자가 본인의 학습 진행정도를 쉽게 파악할 수 있다. 오답이 일정 기준 이상이면 자동으로 재시험이 진행되어, 학습이 부족한 영역에 대해 반복 학습 할 수 있도록 설계했다.

II . ReTest

이 시스템은 영어 단어 시험에서 틀린 문제를 효과적으로 복습할 수 있도록 하는 것을 목적으로 한다. 단순한 일회성 시험이 아닌, 오답을 반복 학습함으로써 학습자의 장기 기억을 강화하고 정확도를 높이는 데 기여한다. 시험이 끝난 후에는 틀린 단어만 선별하여 다시 출제하며, 문제의 순서를 무작위로 섞어 암기 순서에 따른 학습 편향을 최소화 한다. 또한, 사용자의 입력에 따라 즉시 정답 여부를 알려주는 피드백 기능을 제공하여 학습 효과를 극대화한다.

-2) MiniGame

영어 단어 시험이라는 다소 딱딱한 주제에서 벗어나 조금 다른 학습 방향을 제시한다. 사용자가 학습하길 원하는 **DAY**를 입력하면, 해당 시트의 단어를 랜덤으로 5개를 선택해 영어 단어와 뜻을 숫자로 연결하는 방법으로 진행된다. 이는 학생이 영어 단어를 암기하는 과정에서 중간 점검을 할 수 있고, 5개의 적은 단어들을 뜻과 함께 제시하여 비교적 쉽게 풀 수 있도록 설계했다. 이를 통해 학생이 영어 단어를 암기하는데 부담을 덜어주고, 자기 효능감을 높일 수 있기를 기대한다.

2. R 프로그래밍 과정

-[1] R 프로그래밍 과정

-(1) MainTest

I . MainTest

```
# 시험지 로드
#
LoadTest = function(){
  # 사용자 입력 받기
  print("의미를 보고 영단어를 입력하는 시험입니다.")

  repeat{
    day <- as.integer(readline(prompt= "출력하고 싶은 시험지 번호를 입력해주세요. DAY □: "))
    day_sheet <- paste0("DAY", day)

    if(!(day_sheet %in% excel_sheets("C:/Users/82109/Desktop/R.Homework/project/영어 단어장.xlsx"))){
      print("해당 시험지가 존재하지 않습니다.")
      next
    }else{
      break
    }
  }

  # 사용자가 입력한 단어장 불러오기
  cat("시험지를 불러옵니다.\n\n")
  All_words <- read_excel("C:/Users/82109/Desktop/R.Homework/project/영어 단어장.xlsx", sheet = day_sheet)
  sample_words <- All_words[sample(nrow(All_words)),]

  return(list(All_words=All_words,sample_words=sample_words))
}
```

- {1} Load Test

시험지에 대한 설명을 출력한다. repeat문에선 사용자가 원하는 단어장을 입력받고, 엑셀 시트에 존재하지 않는 단어장이라면 다시 입력 문구를 출력한다. 사용자가 엑셀 시트에 존재하는 단어장을 입력하면, 해당 단어장으로 시험지를 만든다. 이는 해당 sheet를 불러와 All_words에 저장하고, sample을 이용해 순서를 섞은 것을 Sample_words에 저장하여, All_words와 Sample_words 리턴함으로써 진행된다.

```
# 메인: 영 단어 시험 진행
#
```

```
MainTest = function(){

  # 단어장 불러오기
  Load_words <- LoadTest()

  # 순서 섞어 시험지 생성
  Test_words <- Load_words$Sample_words
  questions <- Test_words$mean
  solutions <- Test_words$word

  # 정답/오답 저장할 빈 데이터 프레임 생성
  n = length(questions)
  correct = data.frame(correct_word=character(),
                        correct_mean=character(),
                        stringsAsFactors = FALSE)
  wrong = data.frame(wrong_word=character(),
                      wrong_mean=character(),
                      stringsAsFactors = FALSE)
```

- {2} MainTest

- [단어장을 불러와 시험지, 결과노트 생성]

이전 함수의 `Sample_words`를 불러와 `Test_words`에 임시저장한다. 이후, `questions`에 `mean`열을 저장하고, `solutions`에 `word`열을 저장하여 문제지 & 답안지를 생성한다. 이후 `for`문에 사용하기 위해 `n` 벡터에 문항 수를 저장한다. 시험 결과를 저장할 빈 데이터프레임 생성 과정은 다음과 같다. `correct`에 `correct_word`열과 `correct_mean`열을 저장하고, `wrong`에 `wrong_word`열과 `wrong_mean`열을 저장한다. `stringsAsFactors = FALSE`이라는 코드는 `factor`로 저장되는 것을 방지하기 위함이다.

```

# 시험 문제 출력
cat("중단하고 싶다면 stop을 입력해주세요.\n\n")

for(i in 1:n){
  cat(i,"번. ", questions[i])
  answer <- tolower(readline(prompt = "영 단 어 를 입력하세요: "))

  if(answer == solutions[i]){
    cat("정답입니다.\n\n")
    correct <- rbind(correct,
                      data.frame(correct_word = solutions[i],
                                correct_mean = questions[i],
                                stringsAsFactors = FALSE))
  }else if(answer == "stop"){
    break
  }else{
    cat("오답입니다.\n\n")

    wrong <- rbind(wrong,
                   data.frame(wrong_word = solutions[i],
                              wrong_mean = questions[i],
                              stringsAsFactors = FALSE))
  }
}

```

- [시험 문제 출력]

시험을 치르는 중간에 그만두고 싶은 경우 “stop”을 입력하도록 안내한다. for문을 이용해 문항 수 만큼 시험지를 출력한다. 문제는 questions[i]로 출력하고, 사용자의 답안은 readline으로 받아 모두 소문자로 변경(tolower) 후 answer에 저장한다. 채점은 if문으로 진행된다. answer와 solutions[i]가 같으면 “정답입니다”를 출력하고, 이를 correct에 저장한다. 다르다면 “오답입니다”를 출력한 후 이를 wrong에 저장한다. 이는 DataFrame이므로 rbind로 행을 더해가며 저장한다.

```

# 시험 종료 및 결과 출력
print("시험이 종료되었습니다.")
if(answer == "stop"){
  cat("지금까지의 결과를 출력합니다.\n\n")
}

CorrectNum <- nrow(correct)
WrongNum <- nrow(wrong)
Accuracy <- (CorrectNum/(CorrectNum+WrongNum))*100

cat("정답 ", CorrectNum, "개\n",
    "오답 ", WrongNum, "개\n",
    "정답률: ", Accuracy)

```

- [시험 종료 및 결과 출력]

문제를 모두 풀면 “시험이 종료되었습니다.”를 출력한다. if문을 사용해 만약 사용자가 “stop”을 입력한다면 시험을 중단하도록 한다. 이후, 시험을 치른만큼의 결과를 출력한다. 시험 결과를 집계하는 방법은 다음과 같다. CorrectNum은 정답수를 의미하며 correct의 행 개수가 된다. WrongNum는 오답수를 의미하며

`wrong`의 행 개수가 된다. 최종적으로 `Accuracy`에 (정답수)/(정답수 + 오답수)*100로 정답률을 계산하여 저장한다. 이후 시험 결과를 출력한다.

```
# 오답 10개 이상이면 재시험 진행
if (WrongNum >= 10) {
  cat("\n오답이 10개 이상입니다. 자동으로 재시험을 시작합니다.\n\n")

  retry_data <- data.frame(
    단어 = wrong$wrong_word,
    뜻 = wrong$wrong_mean,
    stringsAsFactors = FALSE
  )

  retry_test(retry_data)
}
invisible(list(correct=correct, wrong=wrong))
}
```

- [재시험 시작]

시험 결과, 오답 수가 10개 이상이면 자동으로 재시험을 시작한다. `wrong`의 단어들을 `retry_data`라는 새로운 데이터 프레임으로 저장한다. 단어는 `wrong`의 `word`열이고, 뜻은 `wrong`의 `mean`열이다. `factor`로 저장되는 것을 방지하는 코드도 추가하였다. 이후 `retry_test`라는 재시험 함수를 불러와와 `retry_data`를 넘겨준다. `correct`, `wrong`도 리턴하는데, 이는 프로그램 실행 시 출력되지 않도록 `invisible`로 설정하였다.

II. ReTest

```
# 재시험 진행
# _____

retry_test <- function(retry_data) {

  retry_data <- retry_data[sample(nrow(retry_data)), ]

  cat("재시험을 시작합니다. 아래 뜻에 맞는 영어 단어를 입력하세요.\n\n")

  for (i in seq_len(nrow(retry_data))) {
    meaning <- retry_data$뜻[i] # 뜻
    correct_word <- retry_data$단어[i] # 영어 단어 (정답)

    user_answer <- tolower(readline(paste0(i, ". ", meaning, ": ")))

    if (user_answer == correct_word) {
      cat("정답입니다!\n\n")
    } else {
      cat("틀렸습니다. 정답은: ", correct_word, "\n\n")
    }
  }

  cat("재시험이 종료되었습니다.\n")
}
```

-{1} retry_test

`retry_data`는 재시험 볼 데이터를 `sample`을 사용하여 순서를 랜덤화하였다. `for`문을 `retry_data`의 개수만큼 반복한다. `meaning`은 뜻이다. 이는 `retry_data`의 뜻을 인덱스로 받는다. `correct_word`는 정답이다. 이는 `retry_data`의 원래 영단어를 인덱스로 받는다. `user_answer`로 사용자의 입력을 받는다. `tolower`으로 인해 사용자의 입력 값을 소문자로 변환하여 받으므로 대소문자로 인한 정답오류를 없앤다.

이후, `user_answer`과 앞서 나온 `correct_word`를 비교한다. 정답이면 “정답입니다”를 표시한 후 넘어간다. 오답이면 틀렸다고 표기한 후 올바른 영단어(`correct_word`)를 보여준다. `retry_data` 개수만큼 반복한 후 “재시험이 종료되었습니다”를 표기한 후 종료한다.

-2) MiniGame

```
WordGame = function(){
  print("영단어와 뜻을 숫자로 연결하는 게임입니다.")

  repeat{
    day <- as.integer(readline(prompt= "게임을 시작할 DAY를 입력해주세요. DAY : "))

    day_sheet <- paste0("DAY", day)

    if(!(day_sheet %in% excel_sheets("D:/R_programming/R_programming_statistics/English_exam.xlsx"))){
      cat("해당 DAY가 excel 파일에 존재하지 않습니다.")
      next
    } else {
      break
    }
  }

  cat(day_sheet, "게임을 시작합니다.\n\n")
  day_sheet <- paste0("DAY", day)
  All_words <- read_excel("D:/R_programming/R_programming_statistics/English_exam.xlsx", sheet = day_sheet)
  random_rows <- All_words[sample(nrow(All_words), 5),]
```

- {1}WordGame

게임을 처음 시작할 때 화면에 문제를 출력해주는 함수이다. `repeat`문으로 사용자가 존재하지 않는 `DAY`를 골랐을 때 다시 `DAY` 입력창으로 넘어가기 위한 코드를 구현했다. `day`에는 사용자가 실행할 `DAY`를 숫자로 입력받아 할당한다. `day_sheet`에는 할당받은 숫자와 `DAY`를 합쳐 ‘DAY1’같은 형식으로 만든다. `if`문은 `day_sheet`가 `excel` 파일에 존재할 때 계속 실행하고, `day_sheet`가 `excel` 파일에 존재하지 않을 때 다시 `DAY` 입력창으로 넘어가는 과정을 구현한다. 사용자가 올바른 `DAY`를 입력하면 `All_words`로 사용자가 입력한 `DAY` sheet를 `excel`에서 불러온다. 해당 sheet에서 5개의 랜덤 단어와 뜻을 불러와 `random_rows`에 저장한다.

```
#-----[5개의 단어를 랜덤으로 나열하여 화면에 출력하기 위한 코드]-----

#WORD 순서대로 벡터(숫자 x)
selected_words <- random_rows$word
#MEAN 순서대로 벡터(숫자 x)
selected_means <- random_rows$mean
#MEAN 순서대로 벡터(숫자 0)
selected_means_n <- paste0(1:length(selected_means), '. ', selected_means)

#MEAN 랜덤으로 벡터(숫자 0, 숫자도 랜덤): 답지 작성용
selected_means_n_shuffled <- sample(selected_means_n)
#MEAN 랜덤으로 벡터(숫자 x; 바로 위에 벡터 숫자만 제거)
clean_selected_means_n_shuffled <- sub("^[0-9]+\\.\\s*", "", selected_means_n_shuffled)
#MEAN 랜덤으로 벡터(숫자 0, 숫자는 순서대로): 화면 출력용
selected_means_suffled_n <- paste0(1:length(clean_selected_means_n_shuffled), '. ', clean_selected_means_n_shuffled)

# 최종 화면으로 출력되는 시험지
Game_Screen <-< data.frame(WORD = selected_words, MEAN = selected_means_suffled_n) #Y_ANSWER(), Grade() 에서 nrow를 읽기 위해 전역변수로 바꿈
#-----
```

1) selected_words

WORD
informed
diverse
committed
receipt
fair

3)selected_means_n (2)에서 번호 추가

MEAN
1. 정보에 근거한
2. 다양한
3. 헌신적인
4. 영수증
5. 전시회, 박람회

5)clean_selected_means_n_shuffled (4)의 번호 삭제

MEAN
정보에 근거한
헌신적인
영수증
전시회, 박람회
다양한

2)selected_means

MEAN
정보에 근거한
다양한
헌신적인
영수증
전시회, 박람회

4)selected_means_n_shuffled (3)의 순서 섞기

MEAN
1. 정보에 근거한
3. 헌신적인
4. 영수증
5. 전시회, 박람회
2. 다양한

6)selected_means_shuffled_n (5)에 새로운 번호 부여

MEAN
1. 정보에 근거한
2. 헌신적인
3. 영수증
4. 전시회, 박람회
5. 다양한

- [5개의 단어를 랜덤으로 나열하여 화면에 출력하기 위한 코드]
(헛갈리기 쉬우므로 위의 표와 동일한 번호(예. 1), 2), 3), ...) 을 같이 적었다)
1) selected_words에 random_rows의 word변수에 있는 값을 selected_words에 벡터로 저장한다. 이는 곧 나올 Game_Screen 데이터 프레임 [최종 답지] 에 WORD라는 변수로 들어간다. 2) selected_means에 random_rows의 mean변수에 있는 값을 selected_means에 벡터로 저장한다. 3) selected_means_n에는 selected_means 벡터에 저장된 값들에 순서대로 숫자를 부여한다. 4) selected_means_n_shuffled는 selected_means_n의 요소들을 랜덤으로 뒤섞는다. 이 벡터는 추후 답지 작성용으로 쓰인다. 5) clean_selected_means_n_shuffled에는 뒤섞인 selected_means_n_shuffled의 요소들의 숫자를 제거한다. 6) selected_means_suffled_n는 숫자가 제거된 clean_selected_means_n_shuffled의 요소에 다시 순서대로 숫자를 입력한다. 이는 Game_Screen 데이터 프레임 [최종 답지]에 MEAN이라는 변수로 들어간다.

Game_Screen이라는 데이터 프레임을 생성한다. 이는 WORD라는 변수에 selected_words 벡터를, MEAN이라는 변수에 selected_means_suffled_n라는

벡터를 지정한 데이터 프레임이다. 더욱이 이 **Game_Screen**은 최종 화면으로 출력되는 시험지이다. 참고로 다음에 나올 **Y_ANSWER()**, **Grade()** 함수에서 **nrow**를 읽기 위해 **Game_Screen**은 전역변수로 지정한다.

clean_selected_means_n_shuffled 벡터를 만들 때 사용된 코드를 보충설명해보자면, **sub("^[0-9]+\.\s*", "", selected_means_n_shuffled)**에서 **^**은 문자열의 시작, **[0-9]+**은 하나 이상의 숫자, **\.**는 점(.) 문자, **\s***은 0개 이상의 공백 문자를 의미한다. 즉, 숫자.(공백)을 지우고 의미만 남기는 코드이다.

```
#-----[최종 답지를 만들기 위한 코드]-----
#(빨강색 숫자)
Problem_n <- as.integer(sub("\\.*", "", selected_means_n_shuffled))
#(보라색 숫자)
TRUE_ANSWER <- 1:length(Problem_n)

#Step1
Exam_Answer_not_yet <- data.frame(Problem_n, TRUE_ANSWER)
#Step2,3 ** 최종 답지**
Exam_Answer <- Exam_Answer_not_yet[order(Exam_Answer_not_yet$Problem_n), ] # Grade() 함수에서 사용하기 위해 전역변수로 바꿈
#-----
```

Game—Screen 데이터 프레임

	WORD	MEAN
1	i n f o r m e d	1. 정보에 근거한
2	d i v e r s e	3. 현신적인
3	c o m m i t t e d	4. 영수증
4	r e c e i p t	5. 전시회, 박람회
5	f a i r	2. 다양한

Step 1. 데이터 프레임 생성

TRUE—ANSWER	P r o b l e m — n
1	1
2	3
3	4
4	5
5	2

Step2. Problem—n 기준 정렬

TRUE—ANSWER	Problem—n
1	1
5	2
2	3
3	4
4	5

Step3. Problem—n, TRUE—ANSWER 위치 바꿈

Problem—n	TRUE—ANSWER
1	1
2	5
3	2
4	3
5	4

step3 : 최종 답지

- [최종 답지를 만들기 위한 코드]

(전역변수를 최소화 하기 위해(과부하 방지) WordGame 함수에서 진행한다. 이해를 돕기 위해 위의 표를 (특히 숫자의 배열의 변화와 색깔을) 참고하길 바란다.)

Problem_n은 selected_means_n_shuffled에서 숫자만 가져온다. (빨간색 숫자)

TRUE_ANSWER은 Game_Screen 데이터 프레임의 인덱스를 가져온다. (보라색 숫자)

Exam_Answer_not_yet은 위의 표의 Step1을 실행한다. (보라색 숫자와 빨간색 숫자를 순서대로 나열한 데이터 프레임 생성) Exam_Answer은 Step2,3을 실행한다. (위 데이터 프레임을 빨간색 숫자를 기준으로 정렬하고, 빨간색 숫자와 보라색 숫자의 위치를 바꿈) 이는 최종 답지이다. 이 또한 외부 함수인 Grade()에서 사용하기 위해 전역변수로 바꿨다.

Problem_n에 사용된 코드를 보충설명 하겠다. `sub("\\..*", "", selected_means_n_shuffled))`의 `\\.`은 점(.) 문자, `*`은 점 뒤에 나오는 모든 문자를 의미한다. 즉, 점 이후는 모두 제거하는 코드이다.

```
Y_ANSWER <- function(){
  n <- nrow(Game_Screen)
  Your_answer <- data.frame(Problem_n = 1:n, youranswer = rep(0,n))

  for(i in 1:n){
    cat("\n", i, "번 WORD와 매칭되는 MEAN은 _번 입니다.")
    Your_answer$youranswer[i] <- readline(prompt = "번호를 입력하세요: ")
  }
  you_answer <-< Your_answer # Grade() 함수에서 사용하기 위해 전역변수로 바꿈
  return(you_answer)
}
```

- {2} Y_ANSWER

사용자가 답을 입력하는 함수이다. (사용자가 입력한 답을 **Your_answer**라는 데이터 프레임에 저장) **Your_answer**에는 사용자의 입력값을 저장할 빈 데이터 프레임을 생성하였다. **for** 문으로 **Your_answer\$youranswer[i]**에 사용자의 응답을 각각 번호 저장한다. **you_answer** 또한 외부함수인 **Grade()**에서 사용하기 위해 전역변수로 지정한다.

```
Grade <- function(){
  n <- nrow(Game_Screen)
  for(i in 1:n){
    if(you_answer$youranswer[i] == Exam_Answer$TRUE_ANSWER[i]){
      print(paste0(i, "번 : 정답입니다"))
    } else if (you_answer$youranswer[i] != Exam_Answer$TRUE_ANSWER[i] && you_answer$youranswer[i] >= 1 && you_answer$youranswer[i] <= n){
      print(paste0(i, "번 : 오답입니다"))
    } else {
      print(paste0(i, "번 : 유효하지 않은 값입니다")) # 사용자가 문자나 1~5번 이외의 숫자를 입력했을 때
    }
  }
  cat("\n")
  outcome <- merge(you_answer, Exam_Answer, by = "Problem_n")
  return(outcome)
}
```

- {3} Grade

채점 함수이다. (사용자가 앞서 입력한 값을 실제 정답과 비교하여 정답/오답 여부를 출력하고, 사용자가 입력한 값과 실제 답의 비교를 위해 이 둘을 하나의 데이터 프레임으로 만들어 출력) **for** 문 속 **if**문으로 사용자의 답이 실제 정답과 일치하면, "i번 : 정답입니다", 사용자의 답이 실제 정답과 다르지만, 1~5 사이 숫자인 경우, "i번 : 오답입니다", 사용자의 답이 문자 혹은 1~5 이외의 숫자인 경우, "i번 : 유효하지 않은 값입니다"를 출력한다. **outcome**에는 **Y_ANSWER()** 함수에서 생성한 **you_answer** 데이터 프레임과 **WordGame** 에서 생성한 **Exam_Answer** 데이터 프레임을 **Problem_n**이라는 변수를 기준으로 합친다.

```
MiniGame <- function() {
  repeat {
    # 1회 게임 실행

    print(WordGame()) # 1 함수 실행
    cat("\n")
    print(Y_ANSWER()) # 2 함수 실행
    cat("\n")
    print(Grade()) # 3 함수 실행

    # 사용자에게 계속할지 묻기
    user_input <- tolower(readline(prompt = "\n게임을 계속 하시겠습니까? (계속 하려면 yes, 종료하려면 아무 문자나 입력해주세요.): "))

    if (user_input %in% c("yes")) {
      cat("게임을 다시 시작합니다!\n\n")
    } else {
      cat("게임을 종료합니다. 감사합니다!\n\n")
      break # 반복문 종료
    }
  }
}
```

- {4} MiniGame

실제 미니게임을 실행하는 함수이다. **repeat**문으로 **print(WordGame())**는 화면 출력 함수 실행 및 출력, **print(Y_ANSWER())**는 입력값을 받는 함수의 실행 및 출력, **print(Grade())**는 채점 함수 실행 및 출력을 맡는다. **user_input**으로 사용자에게 계속할지 묻는다. **if**문으로 답이 **yes**이면 게임을 다시 시작한다. (**repeat**문 맨 처음으로 돌아감) **yes**가 아닌 값이라면, 게임을 종료한다.

3) EngWordTest

```
#          기능 선택 (시작 함수)
#_____

EngWordTest <- function(){
  # 프로그램 기능 안내 멘트 출력
  cat("영단어 암기를 도와주는 프로그램입니다.\n")
  cat("원하는 날짜의 시험지를 만들어 영단어를 암기하거나\n")
  cat("미니 게임을 플레이할 수 있습니다.\n\n")

  # 기능 불러오기
  repeat{
    func <- as.integer(readline(prompt = "단어 시험지를 불러오려면 1,\n미니 게임을 플레이하려면 2를 입력해주세요\n\n"))

    if(!(func %in% c(1,2))){
      print("해당 기능이 존재하지 않습니다.")
      next
    }else{
      break
    }
  }

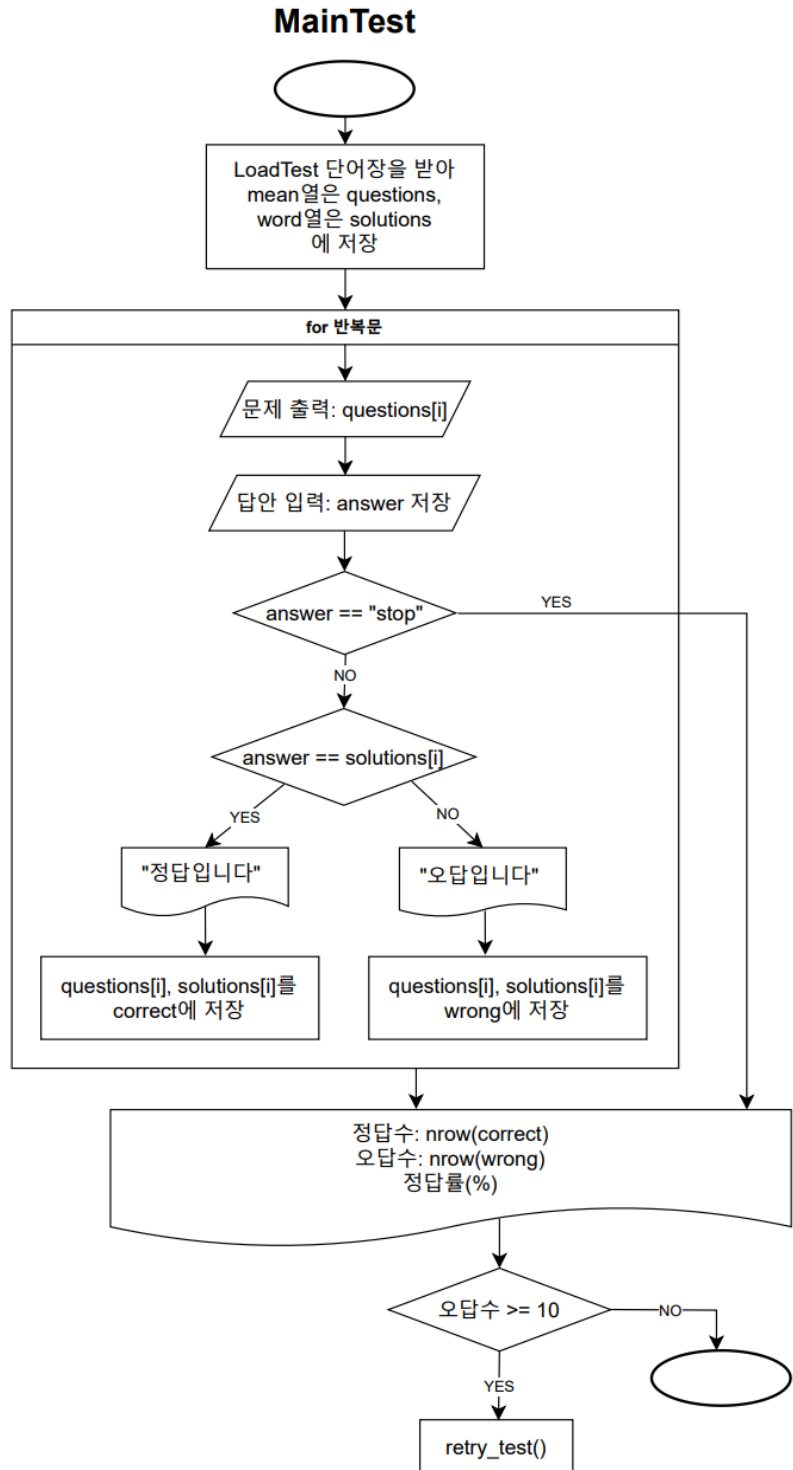
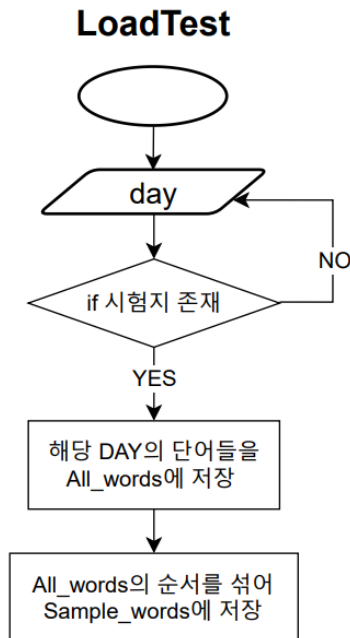
  if(func == 1){
    MainTest()
  }else if(func == 2){
    MiniGame()
  }
}
```

영단어 시험지 프로그램의 최종 실행 함수이다. 프로그램 기능에 대한 안내 멘트를 출력한 후 사용자가 원하는 기능을 불러온다. 사용자의 입력을 **readline**으로 받아 **func**에 저장한다. **func**가 1이나 2가 아니라면 입력 문구 재출력할 수 있도록 **repeat**문을 사용하였다. **if**문으로 사용자가 1번을 선택하면 **MainTest** 함수를 실행하여 영단어 시험을 실행한다. 2번을 선택하면 **MiniGame** 함수 실행하여 미니게임을 실행한다.

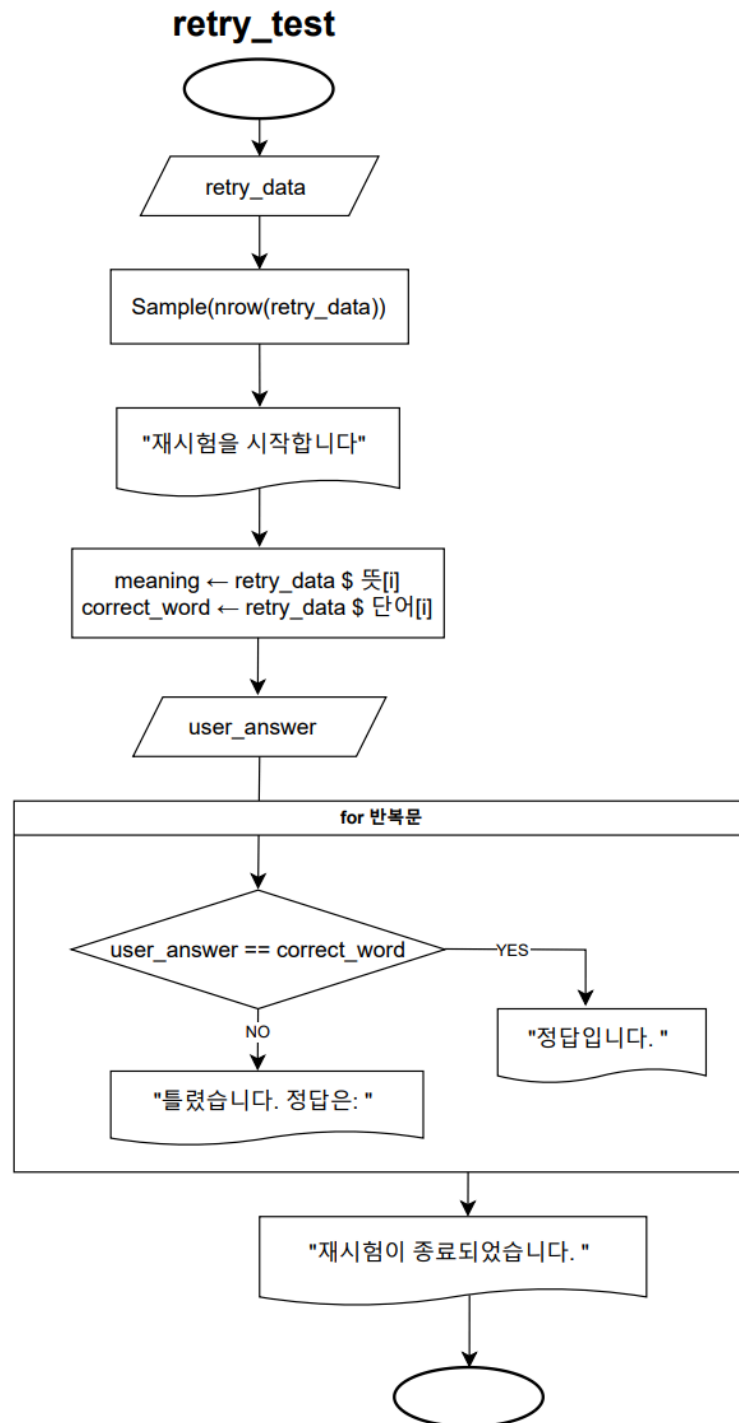
-[2] 알고리즘

-1) MainTest

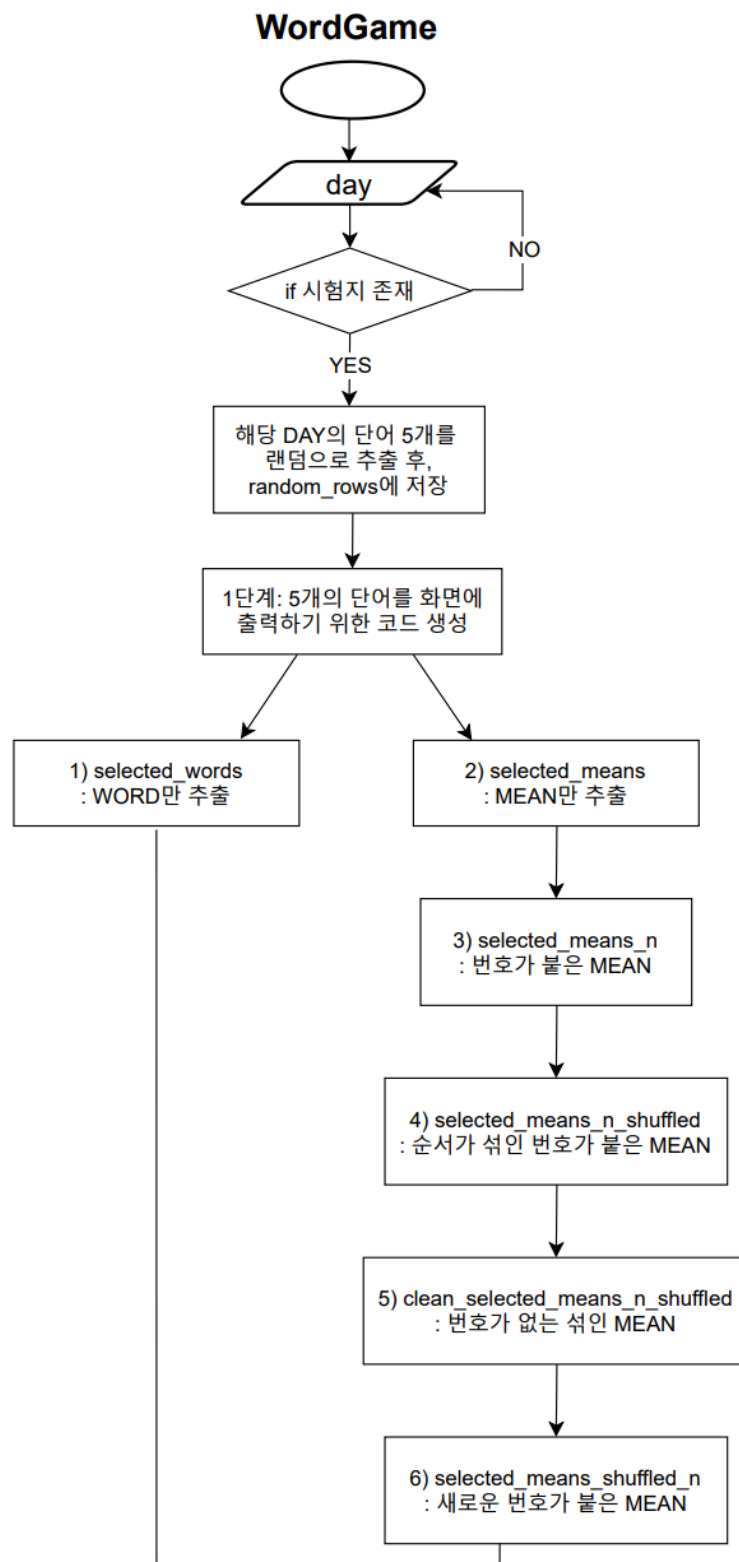
I . MainTest

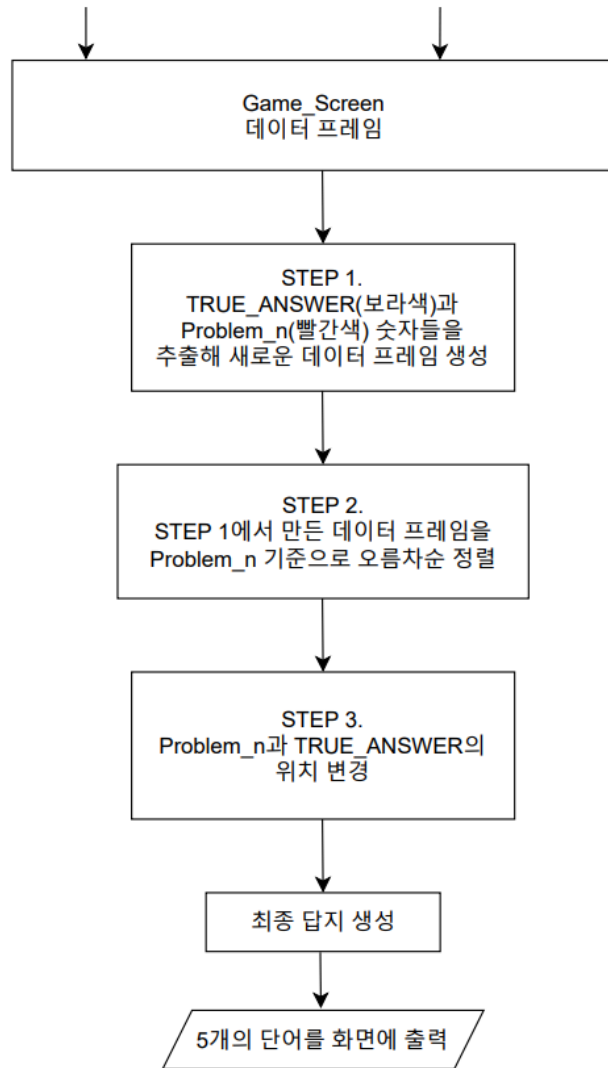


II . ReTest

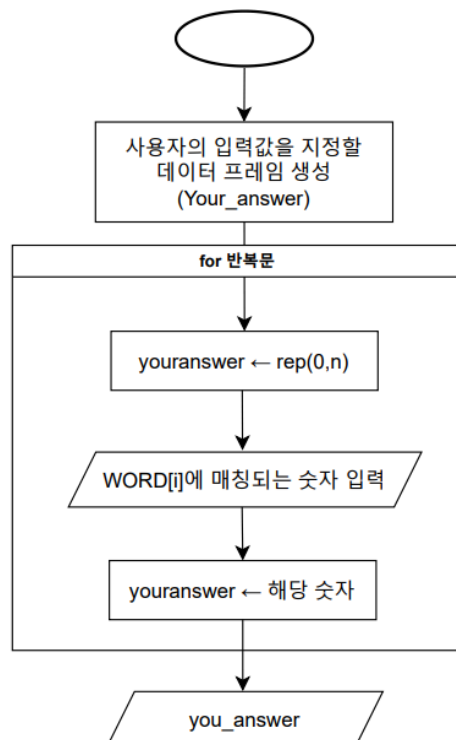


-2) MiniGame

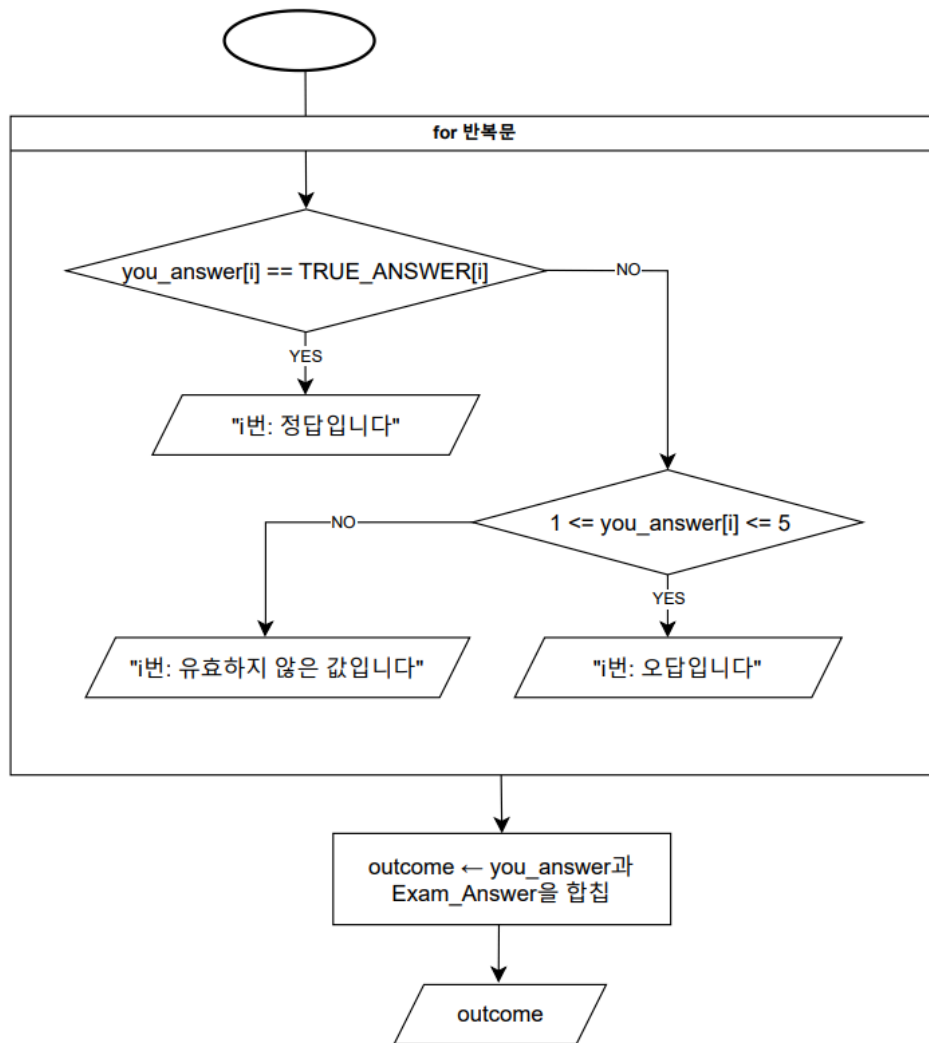




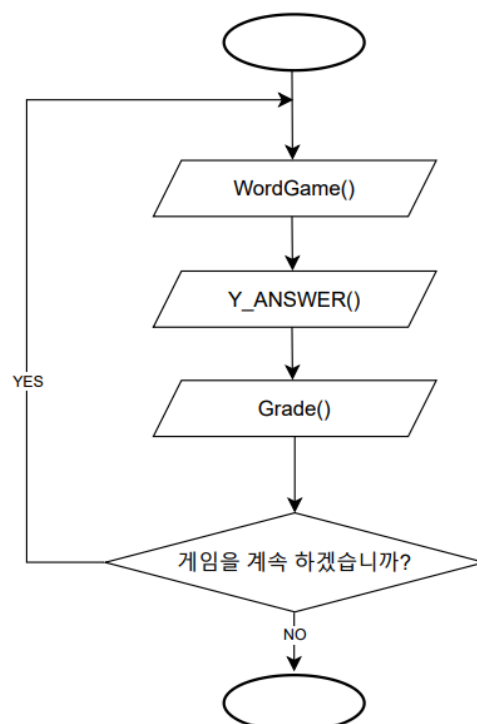
Y_ANSWER



Grade

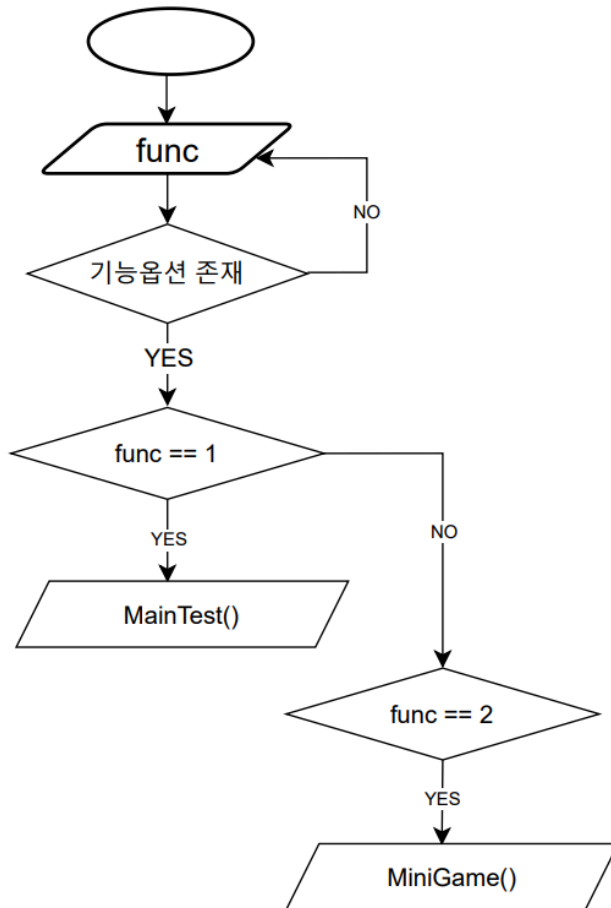


MiniGame



-3) EngWordTest

EngWordTest



3. 사용 설명서

(사용자의 가독성을 위해 '~습니다'체 사용)

```
#          기능 선택 (시작 함수)
#-----

EngWordTest <- function(){

  # 프로그램 기능 안내 멘트 출력
  cat("영단어 암기를 도와주는 프로그램입니다.\n")
  cat("원하는 날짜의 시험지를 만들어 영단어를 암기하거나\n")
  cat("미니 게임을 플레이할 수 있습니다.\n\n")

  # 기능 불러오기
  repeat{
    func <- as.integer(readline(prompt = "단어 시험지를 불러오려면 1,\n미니 게임을 플레이하려면 2를 입력해주세요\n\n"))

    if(!(func %in% c(1,2))){
      print("해당 기능이 존재하지 않습니다.")
      next
    }else{
      break
    }
  }

  if(func == 1){
    MainTest()
  }else if(func == 2){
    MiniGame()
  }
}
```

이 프로그램은 엑셀 파일에 저장된 단어장을 기반으로 영어 단어 시험과 미니 게임을 통해 암기 학습을 돕습니다.

R 콘솔에서 **EngWordTest()** 를 실행하여 프로그램을 시작합니다.
프로그램을 실행하면 다음과 같은 안내 메시지가 출력됩니다.

> EngWordTest()

영단어 암기를 도와주는 프로그램입니다.
원하는 날짜의 시험지를 만들어 영단어를 암기하거나
미니 게임을 플레이할 수 있습니다.

기능을 선택해주세요:

1. 단어 시험지 불러오기
2. 미니 게임 불러오기

번호를 입력하세요:|

1 또는 2 를 입력하여 원하는 기능을 선택할 수 있습니다.

이 때 1과 2 이외의 다른 값을 입력하면

번호를 입력하세요:f

[1] "해당 기능이 존재하지 않습니다."

기능이 존재하지 않다고 뜨며 다시 번호를 입력하도록 되어있습니다.

만약 1을 입력하여 단어 시험지 불러오기를 택하면

[1] "의미를 보고 영단어를 입력하는 시험입니다."

출력하고 싶은 시험지 번호를 입력해주세요. DAY □:

시험 유형을 안내하고 시험을 보고 싶은 day를 입력하도록 합니다.

이 때, day에 들어갈 숫자 이외의 값이 들어가면

출력하고 싶은 시험지 번호를 입력해주세요. DAY □: h

[1] "해당 시험지가 존재하지 않습니다."

출력하고 싶은 시험지 번호를 입력해주세요. DAY □: |

해당 시험지가 존재하지 않으므로 다시 day의 숫자를 입력하도록 되어있습니다.

day에 들어갈 숫자를 입력하고 나면 입력한 day의 단어 시트를 로드합니다.

시험이 시작되고 뜻이 제시되며 사용자가 단어를 입력합니다.

출력하고 싶은 시험지 번호를 입력해주세요. DAY □: 1

시험지를 불러옵니다.

중단하고 싶다면 stop을 입력해주세요.

1 번. 파악하다, 확인하다

영단어를 입력하세요: |

사용자가 영어 단어를 입력하고 enter를 쳐서 넘어가면

정답일 시 '정답입니다.' 라고 표시된 후 다음 문제로 넘어갑니다.

오답일 시 '오답입니다.' 라고 표시된 후 다음 문제로 넘어갑니다.

1 번. 파악하다, 확인하다
영단어를 입력하세요: **identify**
정답입니다.

2 번. 전시회, 박람회
영단어를 입력하세요: **f**
오답입니다.

만약 시험을 도중에 그만두고 싶을 경우 'stop' 을 입력하여 시험을 종료할 수 있습니다.

중단하고 싶다면 **stop**을 입력해주세요.

1 번. 신고하다
영단어를 입력하세요: **stop**
[1] "시험이 종료되었습니다."

day 시트에 있는 단어 개수만큼 입력을 마치면 시험의 결과를 출력해줍니다.
시험의 결과는 정답의 개수, 오답의 개수, 정답률을 표시해줍니다.

[1] "시험이 종료되었습니다."
정답 9 개
오답 23 개
정답률: 28.125

만약 시험에서 오답이 10개 이상이면 자동으로 재시험이 시작됩니다.
오답만 무작위로 다시 출제됩니다.

재시험 시,
입력한 값이 정답이면 '정답입니다!' 가 표시되고
입력한 값이 오답이면 '틀렸습니다. 정답은: 올바른 영단어' 가 표시됩니다.
재시험을 시작합니다. 아래 뜻에 맞는 영어 단어를 입력하세요.

1. 기여, 공헌: **d**
틀렸습니다. 정답은: **contribution**

2. 다양한: **diverse**
정답입니다!

본 시험의 오답의 개수만큼 반복 후 '재시험이 종료되었습니다.' 문구 표시 후 재시험 프로그램이 종료됩니다.

만약 2를 입력하여 미니 게임 불러오기를 택하면

[1] "영 단어와 뜻을 숫자로 연결하는 게임입니다."
게임을 시작할 DAY를 입력해주세요. DAY □: |

미니 게임 시작을 안내 한 후 day의 숫자를 입력하도록 합니다.

게임을 시작할 DAY를 입력해주세요. DAY □: f
해당 DAY가 excel 파일에 존재하지 않습니다.
게임을 시작할 DAY를 입력해주세요. DAY □: 1
DAY1 게임을 시작합니다.

이 때 day에 들어갈 숫자 이외의 것을 입력하면 excel 파일에 존재하지 않는다고 알려준 후 다시 day의 숫자를 입력하도록 합니다.

올바른 숫자를 입력하면 게임을 시작한다는 문구가 뜨게 됩니다.

DAY1 게임을 시작합니다.

	WORD	MEAN
1	diverse	1. 다양한
2	committed	2. 헌신적인
3	exceptional	3. 해결하다
4	resolve	4. 연례의, 매년의
5	annual	5. 특별한, 비범한

게임이 시작되면 day내의 무작위 단어 5개와 섞인 뜻 목록이 출력됩니다.

1 번 WORD와 매칭되는 MEAN은 _번 입니다.
번호를 입력하세요: 1

2 번 WORD와 매칭되는 MEAN은 _번 입니다.
번호를 입력하세요: 2

3 번 WORD와 매칭되는 MEAN은 _번 입니다.
번호를 입력하세요: 5

4 번 WORD와 매칭되는 MEAN은 _번 입니다.
번호를 입력하세요: 1

5 번 WORD와 매칭되는 MEAN은 _번 입니다.
번호를 입력하세요: 4

매칭 입력을 하도록 문구가 뜨고 사용자가 번호를 입력합니다.

이 때 1~5숫자 외의 다른 숫자나 문자를 입력하거나 아예 값을 입력하지 않으면

	Problem_n	youranswer
1	1	9
2	2	10
3	3	
4	4	ㅅ
5	5	y

[1] "1번 : 유효하지 않은 값입니다"
[1] "2번 : 오답입니다"
[1] "3번 : 유효하지 않은 값입니다"
[1] "4번 : 유효하지 않은 값입니다"
[1] "5번 : 유효하지 않은 값입니다"

5번의 입력 후 결과로 '유효하지 않은 값입니다' 가 출력됩니다.

숫자를 입력하게 되면

	Problem_n	youranswer
1	1	1
2	2	2
3	3	5
4	4	1
5	5	4

문제의 번호와 사용자가 입력한 문제의 답이 표시되고

[1] "1번 : 정답입니다"
[1] "2번 : 정답입니다"
[1] "3번 : 정답입니다"
[1] "4번 : 오답입니다"
[1] "5번 : 정답입니다"

	Problem_n	youranswer	TRUE_ANSWER
1	1	1	1
2	2	2	2
3	3	5	5
4	4	1	3
5	5	4	4

각 번호마다 정답인지 오답인지 문구를 표시해준 후
문제의 번호와 사용자가 입력한 답과 정답의 숫자를 표시하여 비교할 수 있게 합니다.

이 과정을 끝내면 게임의 반복 여부를 묻습니다.

게임을 계속 하시겠습니까? (계속 하려면 **yes**, 종료하려면 아무 문자나 입력해주세요.): |

yes를 입력하게 되면 다시 **day**의 숫자를 입력하게 되고 **yes**외의 문자를 입력하게 되면 게임을 종료하게 됩니다.

4. 마무리 및 해당 프로그램을 통한 영어 학습의 의의

MainTest와 **ReTest**, 그리고 **MiniGame**을 통해 학생들이 영어 단어 암기의 부담을 덜 수 있기를 기대한다. 언제든 반복 학습이 가능하며 재시험을 통해 자신의 학습 정도를 정확한 수치로 판단하여, 영단어 학습에 대한 메타인지를 향상할 수 있을 것이다. 특히 게임을 통한 학습은 학생들에게 영어 단어 학습 진입 장벽을 낮춰주어 학업 성취도가 높지 않은 학생에게도 충분히 적용 가능하다. 모든 프로그램이 성적 상위권 학생들부터 하위권 학생들까지 모두 영어 단어 학습에 도움을 받을 수 있다는 점에서 의의가 있다. 더욱이 영어 과외를 하는 이화여자대학교 벳들도 해당 프로그램을 수업에서 바로 활용할 수 있기에 높은 효용성을 지닌다. 과외 학생과 선생님 모두에게 긍정적인 효과를 가져다 줄 수 있을 것이다.